

Competitive Programming

Section 36 Editorial: Fenwick Trees

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
2	A. Point Add Range Sum	4
2.1	Restatement	4
2.2	Algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	5
3	B. Inversion Count	7
3.1	Restatement	7
3.2	Algorithm	7
3.3	Correctness proof	7
3.4	Complexity	7
3.5	Full C++ solution	7
3.6	Full Python solution	8
4	C. Range Add Point Query	10
4.1	Restatement	10
4.2	Algorithm	10
4.3	Correctness proof	10
4.4	Complexity	10
4.5	Full C++ solution	10
4.6	Full Python solution	11
5	Testing discussion	13

1 Editorial overview

Problem	Main idea
A. Point Add Range Sum	See full algorithm and code below.
B. Inversion Count	See full algorithm and code below.
C. Range Add Point Query	See full algorithm and code below.

2 A. Point Add Range Sum

2.1 Restatement

Maintain an array. Operation 1 $i\ x$ adds x to $a[i]$. Operation 2 $l\ r$ asks for the sum of $a[l..r]$.

2.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

2.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

2.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<long long> bit;
```

```

Fenwick(int n) : n(n), bit(n + 1, 0) {}
void add(int i, long long delta) {
    for (; i <= n; i += i & -i) bit[i] += delta;
}
long long sum(int i) const {
    long long res = 0;
    for (; i > 0; i -= i & -i) res += bit[i];
    return res;
}
long long range_sum(int l, int r) const {
    return sum(r) - sum(l - 1);
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    Fenwick bit(n);
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        bit.add(i, x);
    }
    while (q--) {
        int typ;
        cin >> typ;
        if (typ == 1) {
            int i;
            long long x;
            cin >> i >> x;
            bit.add(i, x);
        } else {
            int l, r;
            cin >> l >> r;
            cout << bit.range_sum(l, r) << '\n';
        }
    }
    return 0;
}

```

2.6 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n):

```

```

        self.n = n
        self.bit = [0] * (n + 1)
    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
    def sum(self, i):
        res = 0
        while i > 0:
            res += self.bit[i]
            i -= i & -i
        return res
    def range_sum(self, l, r):
        return self.sum(r) - self.sum(l - 1)

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    bit = Fenwick(n)
    for i, x in enumerate(a, 1):
        bit.add(i, x)
    idx = 2 + n
    out = []
    for _ in range(q):
        typ = data[idx]
        idx += 1
        if typ == 1:
            i, x = data[idx], data[idx + 1]
            idx += 2
            bit.add(i, x)
        else:
            l, r = data[idx], data[idx + 1]
            idx += 2
            out.append(str(bit.range_sum(l, r)))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

3 B. Inversion Count

3.1 Restatement

Given an array, count pairs $i < j$ with $a[i] > a[j]$. Values may repeat and may be large.

3.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

3.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

3.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;
    Fenwick(int n) : n(n), bit(n + 1, 0) {}
};
```

```

void add(int i, int delta) {
    for (; i <= n; i += i & -i) bit[i] += delta;
}
int sum(int i) const {
    int res = 0;
    for (; i > 0; i -= i & -i) res += bit[i];
    return res;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n), vals;
    for (long long &x : a) {
        cin >> x;
        vals.push_back(x);
    }
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    Fenwick bit((int)vals.size());
    long long inv = 0;
    for (int i = 0; i < n; i++) {
        int rank = int(lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin()) + 1;
        inv += i - bit.sum(rank);
        bit.add(rank, 1);
    }
    cout << inv << '\n';
    return 0;
}

```

3.6 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)
    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
    def sum(self, i):
        res = 0
        while i > 0:

```

```

        res += self.bit[i]
        i -= i & -i
    return res

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    a = data[1:1 + n]
    values = {x: i + 1 for i, x in enumerate(sorted(set(a)))}
    bit = Fenwick(len(values))
    inv = 0
    seen = 0
    for x in a:
        rank = values[x]
        inv += seen - bit.sum(rank)
        bit.add(rank, 1)
        seen += 1
    print(inv)

if __name__ == "__main__":
    main()

```

4 C. Range Add Point Query

4.1 Restatement

Maintain an array. Operation 1 $l\ r\ x$ adds x to every element in $[l, r]$. Operation 2 i asks for the current value of $a[i]$.

4.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

4.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

4.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<long long> bit;
```

```

Fenwick(int n) : n(n), bit(n + 2, 0) {}
void add(int i, long long delta) {
    for (; i <= n; i += i & -i) bit[i] += delta;
}
long long sum(int i) const {
    long long res = 0;
    for (; i > 0; i -= i & -i) res += bit[i];
    return res;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    Fenwick bit(n + 1);
    long long prev = 0;
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        bit.add(i, x - prev);
        prev = x;
    }
    while (q--) {
        int typ;
        cin >> typ;
        if (typ == 1) {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            bit.add(l, x);
            bit.add(r + 1, -x);
        } else {
            int i;
            cin >> i;
            cout << bit.sum(i) << '\n';
        }
    }
    return 0;
}

```

4.6 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n):

```

```

        self.n = n
        self.bit = [0] * (n + 2)
    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
    def sum(self, i):
        res = 0
        while i > 0:
            res += self.bit[i]
            i -= i & -i
        return res

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    bit = Fenwick(n + 1)
    prev = 0
    for i, x in enumerate(a, 1):
        bit.add(i, x - prev)
        prev = x
    idx = 2 + n
    out = []
    for _ in range(q):
        typ = data[idx]
        idx += 1
        if typ == 1:
            l, r, x = data[idx], data[idx + 1], data[idx + 2]
            idx += 3
            bit.add(l, x)
            bit.add(r + 1, -x)
        else:
            i = data[idx]
            idx += 1
            out.append(str(bit.sum(i)))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

5 Testing discussion

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.